BCSE203E - Web Programming

Digital Assignment - IV

Winter Semester 2025-2026

Topic: Team-Based Web Application Development

Project Title: NestQuest - Smart Roommate Compatibility Finder

Team Name: Questers

Team Composition & Role Distribution:

- **Anjini Pandey (24BCE0714)** *Role:* Frontend Developer & UI/UX Designer
- **Yash Pradhan (24BCE0702)** *Role:* Frontend Developer & Logic Architect

Project Repository & Live Deployment

As per the evaluation criteria, the project source code and review documentation for NestQuest have been pushed to the faculty-provided GitHub repository.

However, because GitHub Pages hosting is restricted on the faculty-provided repository, we have deployed the live application via a personal GitHub repository. To present our work professionally and demonstrate consistent progress, we integrated this live deployment into a comprehensive coursework repository that also houses our Web Programming lab tasks from throughout the semester.

- **Main GitHub Repository Link (Official Classroom Repo for Source Code):**

  https://github.com/JayakumarClassroom/daily-task-yoswinner

- **Coursework and project repository Link (Lab Tasks & Final Project) :** https://github.com/yoswinner/Web-Programming-FS-25-26

- **GitHub Pages Live Link (NestQuest Deployment & Coursework):**

  https://yoswinner.github.io/Web-Programming-FS-25-26/

Table of Contents

1. Project Overview & Innovation
2. Collaboration & Role Distribution
3. Technical Architecture & Proficiency
4. Mandatory Form Validations Implemented
5. Comprehensive Project Planning & SDLC
6. UI/UX Design & Usability
7. Interface Design & Implementation Walkthrough
8. Conclusion & Future Scope

1. Project Overview & Innovation

Problem Statement

University students and young professionals frequently face the challenge of incompatible roommate pairings. Traditional hostel or apartment allocations are often randomized or based solely on brief interactions, leading to conflicts over lifestyle differences (e.g., sleep schedules, cleanliness).

The NestQuest Solution

NestQuest is an innovative, frontend-based web application designed to solve this real-world problem. Instead of randomized pairings, NestQuest introduces a data-driven compatibility algorithm. Users input their specific lifestyle preferences, and the application mathematically evaluates them against a database of potential roommates.

Relevance

The project utilizes modern web development trends, including a "Tinder-style" swipe interface for intuitive matching, simulating modern progressive web apps (PWAs) entirely through Vanilla JavaScript, HTML5, and CSS3 without relying on heavy frontend frameworks.

2. Collaboration & Role Distribution

To ensure a substantive and collaborative effort suitable for this assignment's scope, the workload was distributed effectively to utilize the strengths of both team members.

- **Yash Pradhan (24BCE0702) - Frontend Developer & Logic Architect:** Led the development of core JavaScript logic and application state, including strict form validations, data-scoring algorithms, and dynamic dashboard sorting and filtering. Also played a key role in UI development, contributing to layout design, theme integration (Light/Dark mode), and ensuring a cohesive and intuitive user interface. Additionally, architected a persistent client-side state management system utilizing the browser's LocalStorage API to maintain user session data without a backend. He also engineered the interactive swipe mechanics, using raw DOM events and dynamic CSS transforms to deliver a seamless and tactile user experience.

- **Anjini Pandey (24BCE0714) - Frontend Developer & UI/UX Designer:**
  Led the visual architecture and user experience design, implementing the CSS Glassmorphism aesthetic and designing dynamic responsive grids. Also contributed to core JavaScript logic, assisting in interactive features like drag-and-drop mechanics and enhancing the responsiveness of UI components. Additionally, structured the informative "About" page layout. She further engineered a seamless Light/Dark mode theme engine utilizing dynamic CSS Custom Properties for instant DOM recoloring. Moreover, she championed web accessibility by enforcing native HTML5 form constraints and crafting tactile micro-interactions to elevate the overall user journey.

  (Note: Both members collaborated equally on the overarching HTML structure and ensuring responsive design integrity across all viewports).

3. Technical Architecture & Proficiency

The application was built emphasizing code quality and technical proficiency:

- **State Management:** Fully utilizes browser LocalStorage to maintain user session state (nestquest_loggedIn), store user profile configurations (nestquest_profile), and track algorithm queues (match_queue, accepted_users, rejected_users).
- **Custom Routing:** Engineered a custom Vanilla JS router that listens to the DOM structure to execute specific logic only when relevant, simulating a Single Page Application (SPA) feel while maintaining distinct HTML files.
- **Algorithmic Evaluation:** A custom function evaluateMatch() applies a 25% weight to four core lifestyle pillars to generate a definitive percentage match and dynamically generates the reasoning behind the match for the user to read.

4. Mandatory Form Validations Implemented

Ensuring data integrity and an excellent user experience was paramount. We successfully engineered three distinct, heavily validated forms to meet the strict technical requirements of the assignment:

Form 1: Authentication & Security (login.html)

- **Purpose:** Handles user sign-up and login securely.
- **Validation Logic:**
  - Enforces strict email domain validation (inputs *must* end in @gmail.com).
  - Utilizes custom Regex validation to enforce a strict password policy: Minimum 8 characters, requiring at least one number, one uppercase letter, and one special symbol.
  - **UX Enhancement:** A dynamic, color-coded password strength meter provides real-time feedback to the user via DOM manipulation.

Form 2: Profile & Demographic Setup (index.html)

- **Purpose:** Captures the user's details and exact roommate preferences to feed the algorithm.
- **Validation Logic:**
  - Utilizes HTML5 constraints to restrict the age input explicitly between 16 and 50 (min="16" max="50").
  - Employs JavaScript event listeners to intercept the form submission, ensuring all drop-down preferences (Sleep, Cleanliness, Social, Study) are explicitly selected, preventing null data from breaking the matching algorithm.

Form 3: Communication Integration (dashboard.html)

- **Purpose:** Allows a user to instantly reach out to an accepted match via email.
- **Validation Logic:**
  - Rendered as an interactive modal overlay to prevent navigating away from the dashboard.
  - Automatically fetches and locks the selected match's email address as a read-only field to prevent user error.
  - Requires non-empty inputs for both Subject and Body.
  - Safely encodes the inputs using encodeURIComponent() before passing the data to the native mail client via a mailto: protocol.

5. Comprehensive Project Planning & SDLC

To execute this project efficiently, our team adopted a structured Software Development Life Cycle (SDLC) workflow, breaking the vision into actionable micro-phases:

Phase 1: Conceptualization & UI/UX Architecture

Before implementing the frontend logic, we established a strict design system to ensure visual consistency across all five views.

- **Design System Definition:** Selected a "Neon Glassmorphism" aesthetic to appeal to a modern student demographic. Defined global CSS variables for primary colors (--neon-pink, --neon-purple, --glass-bg) to maintain consistency.
- **Component Mapping:** Designed the universal Navigation Bar, ensuring the profile avatar dynamically reflects the user's selected emoji across all pages.
- **Theme Engine Planning:** Outlined the logic for a CSS class-based toggle system (body.light-theme) that flips dark transparent backgrounds to light transparent backgrounds instantly.

Phase 2: State Management & Application Routing

Since the project relies purely on frontend technologies, we had to engineer a way to store data and handle page transitions smoothly.

- **Local Database Architecture:** Designed a JSON-based dummy database of 30 unique student profiles, storing attributes like name, hobbies, sleep schedule, and cleanliness standards.
- **State Persistence:** Mapped out the LocalStorage keys required to save user sessions: nestquest_loggedIn, nestquest_profile, match_queue, accepted_users, and rejected_users.
- **DOM Routing Setup:** Engineered a custom Javascript router utilizing window.location.pathname and element detection (e.g., document.getElementById('loginForm') !== null) to execute specific scripts only when the relevant HTML page is loaded, preventing console errors.

Phase 3: Form Engineering & Security Validation

Addressing the core assignment requirement of implementing three validated forms to ensure data integrity.

- **Form 1 (Authentication):** Planned a Regex-based validation system for the password field to enforce strict security (minimum 8 characters, one number, one uppercase letter, one

special symbol) and mapped it to a dynamic visual strength meter.

- **Form 2 (Profile Setup):** Designed a two-step form flow. Step 1 handles basic demographics with HTML5 constraints (Age min="16" max="50"). Step 2 handles roommate preferences, requiring explicit dropdown selections before submission.
- **Form 3 (Communication):** Outlined a modal-based email composer that dynamically fetches the selected match's email and user name, sanitizing the input using encodeURIComponent before pushing it to the native mail client.

Phase 4: The Core Matching Algorithm

We needed a mathematical approach to determine roommate compatibility rather than showing random profiles.

- **Scoring Logic:** Assigned a 25% weight to each of the four core lifestyle pillars (Sleep Schedule, Cleanliness, Study Vibe, Social Energy).
- **Evaluation Function:** Designed the evaluateMatch(prefs, dummyUser) function to compare the user's LocalStorage preferences against the active array object.
- **Dynamic Feedback:** Engineered the algorithm to push specific string reasons (e.g., "Agrees on cleanliness") into an array, which is then rendered on the UI so the user understands exactly *why* they were matched.

Phase 5: Interactive Swipe Mechanics

To modernize the user experience, we planned a complex interactive component relying on raw DOM mouse events.

- **Event Listeners:** Implemented mousedown, mousemove, and mouseup tracking to calculate the X-axis delta (clientX - startX).
- **Transform Logic:** Applied dynamic CSS transform: translateX() rotate() properties based on the mouse drag distance.
- **Threshold Triggers:** Set an absolute threshold (> 150px or < -150px) to automatically trigger the accept or reject logic, push the user ID to the respective LocalStorage array, and apply the fade-out CSS keyframe animation.

Phase 6: Dashboard Data Filtering & Rendering

The final phase required building a complex data presentation layer.

- **Tab State Management:** Planned an event listener system to switch the active UI tab (Connected, Passed, Suggested) and trigger a re-render of the grid.
- **Array Manipulation:** Utilized Javascript .filter() to separate accepted vs. rejected IDs, and .map() to attach the calculated match scores to each object dynamically.
- **Multi-Parameter Sorting:** Engineered a toolbar utilizing .sort() to organize the rendered arrays alphabetically, by age, or by highest match percentage.

Development Timeline & Sprints

Sprint	Focus Area	Key Deliverables
Sprint 1	Foundation	CSS variables, Animated Background, Glassmorphism UI, HTML outlines.
Sprint 2	Security & State	Login form Regex validation, LocalStorage initialization, Custom JS Router.
Sprint 3	Data & Logic	Profile creation validation, 30-user JSON database, Weighted matching algorithm.
Sprint 4	Interactivity	DOM Mouse events for Swipe cards, CSS Keyframe animations.
Sprint 5	Data Rendering	Dashboard grid mapping, array filtering, multi-parameter sorting.
Sprint 6	Polish & Integration	Light/Dark theme toggle, Email composer modal, Cross-browser bug fixing.

6. UI/UX Design & Usability

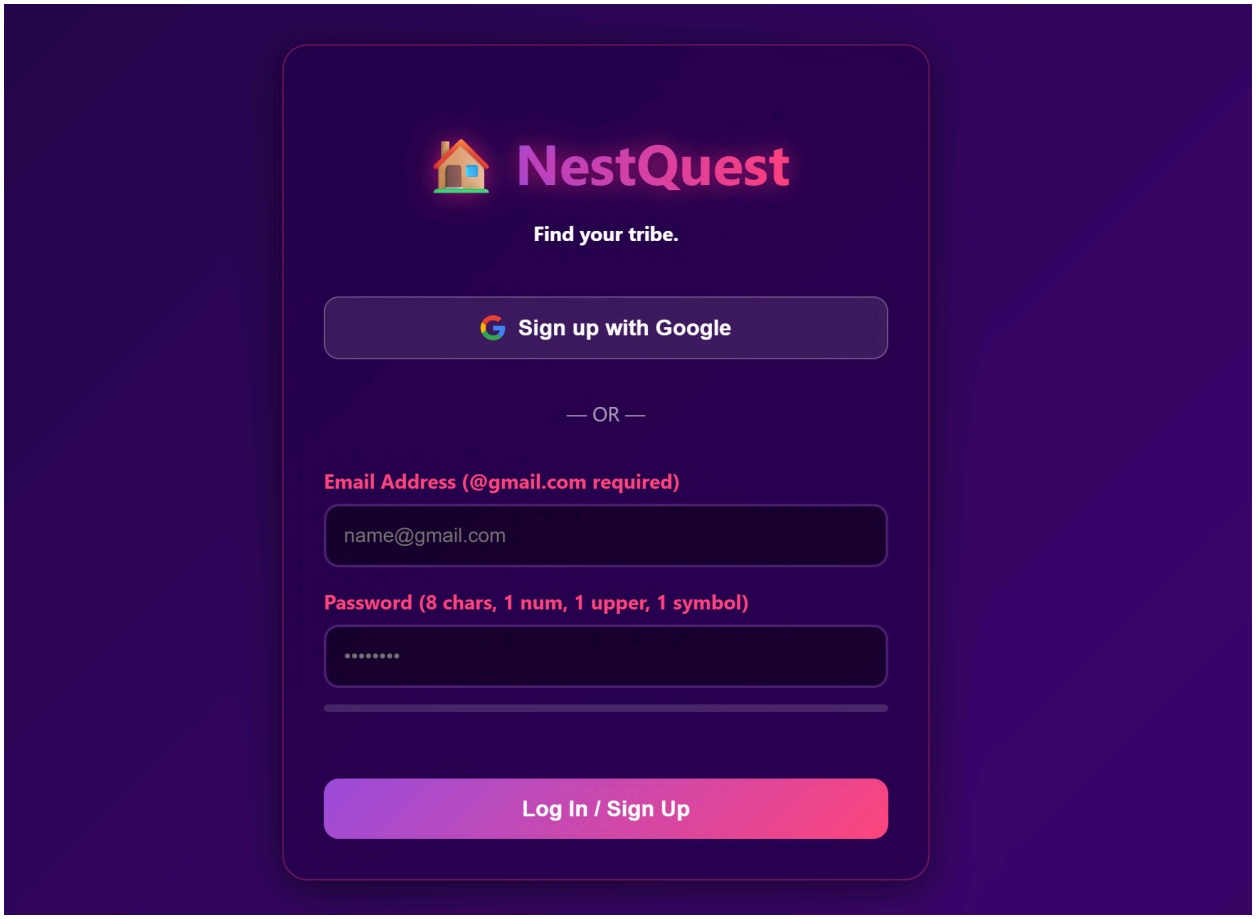
The aesthetic appeal and intuitiveness of NestQuest were prioritized from day one.

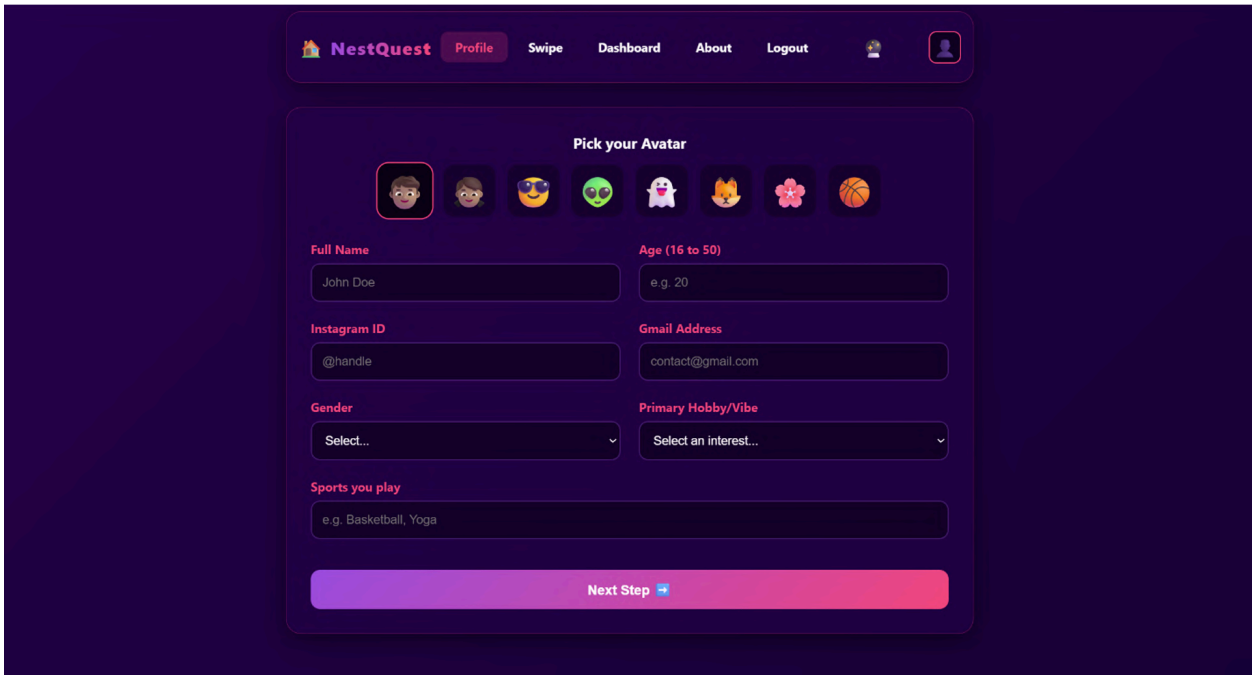
- **Responsive Design:** Utilized CSS Grid and Flexbox to ensure the application adapts flawlessly across desktop, tablet, and mobile viewports.
- **Glassmorphism Aesthetic:** Employed CSS backdrop-filter: blur() paired with semi-transparent rgba backgrounds and animated gradient underlays to create a premium, modern feel.
- **Micro-interactions:** Added CSS keyframe animations (fade-ins, throw-left, throw-right) to give the application a tactile, responsive feel when users interact with the swipe cards or submit forms.
- **Accessibility:** Ensured high color contrast (especially in the Dark Theme) and provided a distinct Light Theme alternative for users who prefer brighter interfaces.

7. Interface Design & Implementation Walkthrough

This section provides a visual walkthrough of the NestQuest application, highlighting the core technical implementations and the "Glassmorphism" design system developed for this project.

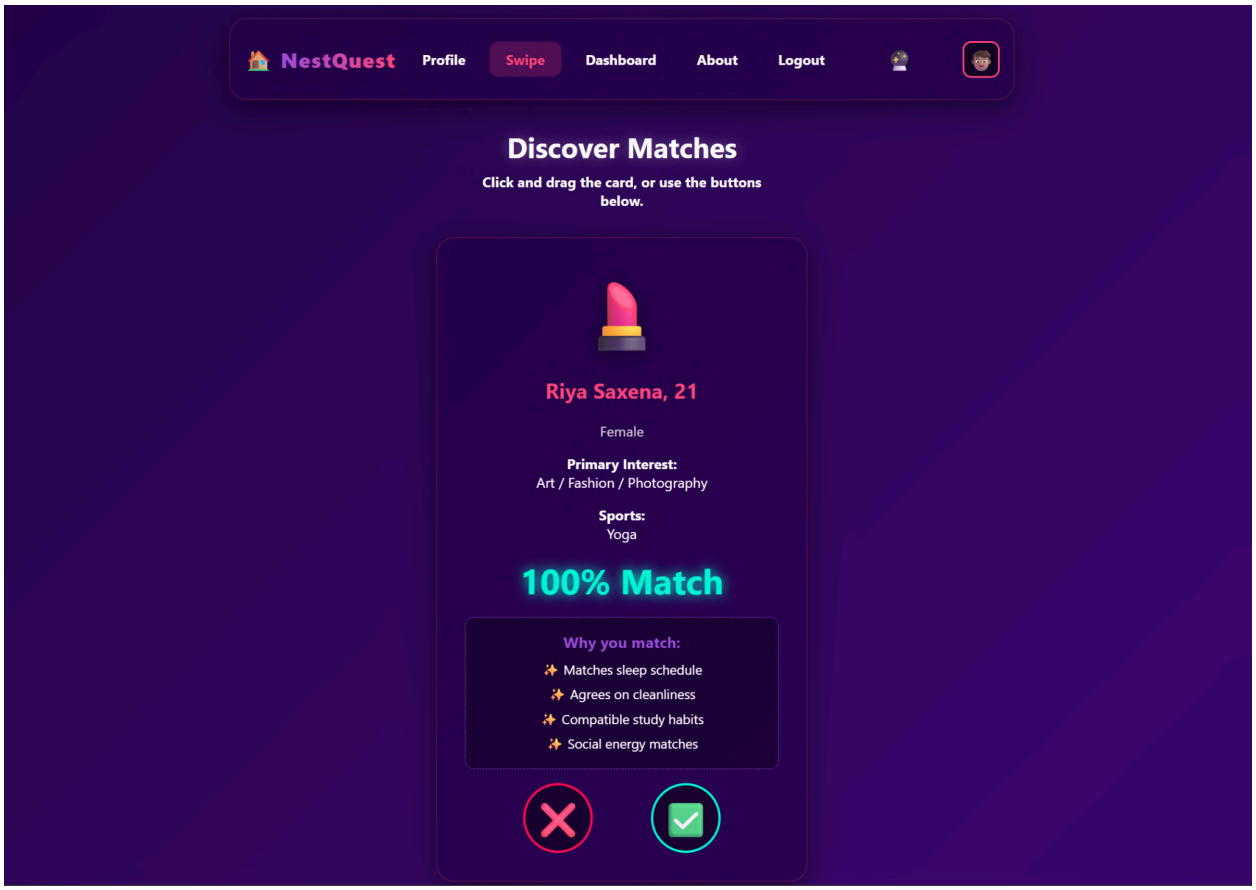
7.1 Authentication & The Glassmorphism Landing Page





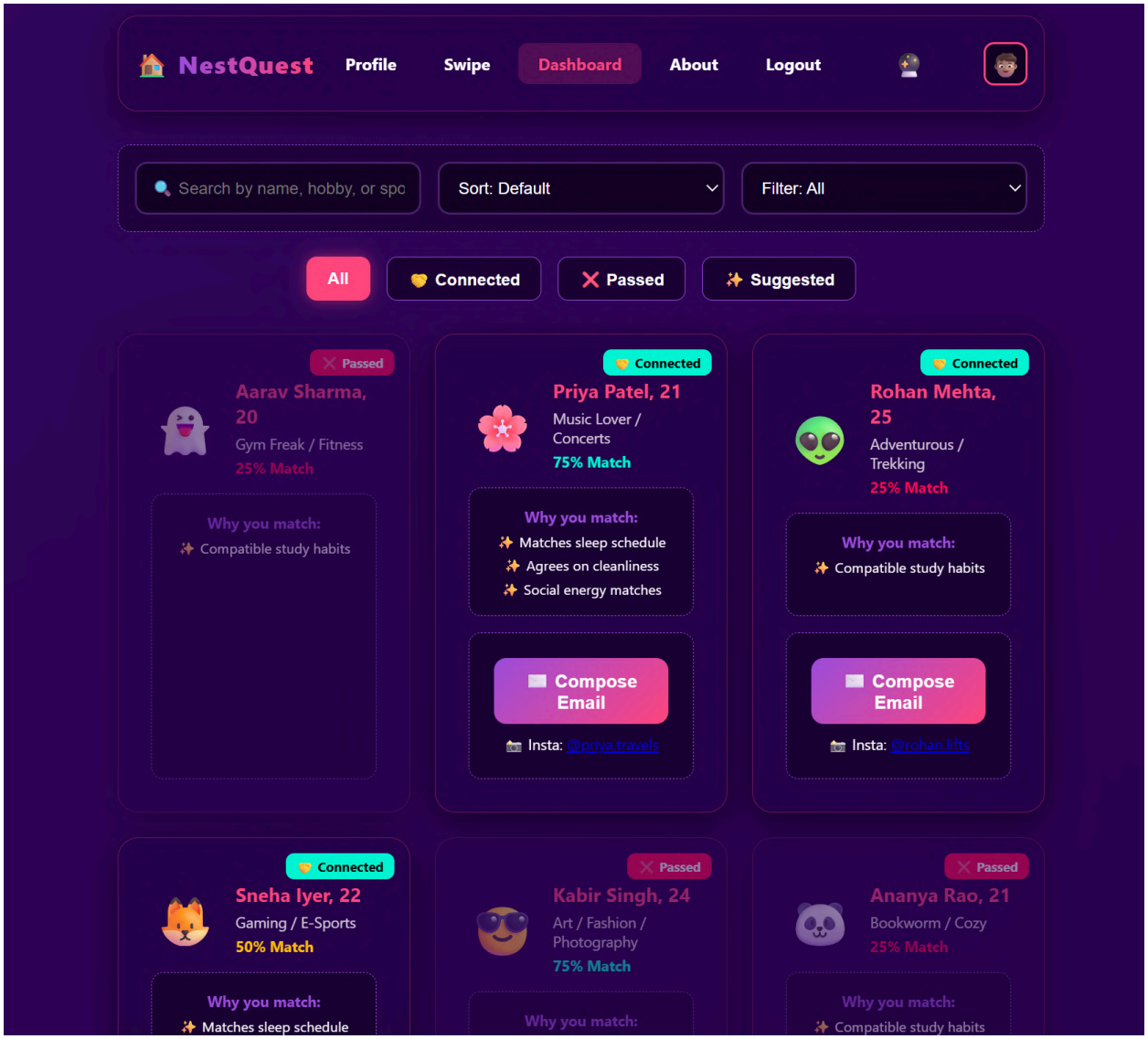
- **Design Implementation:** The landing page utilizes the "Glassmorphism" aesthetic, achieved through CSS backdrop-filter: blur(15px) and semi-transparent rgba background colors.
- **Technical Feature:** This serves as the first of our three validated forms. The login system utilizes JavaScript Regex to validate email domains and password strength before allowing access to the application state.

7.2 Interactive Swipe Mechanics (Core Matching Logic)



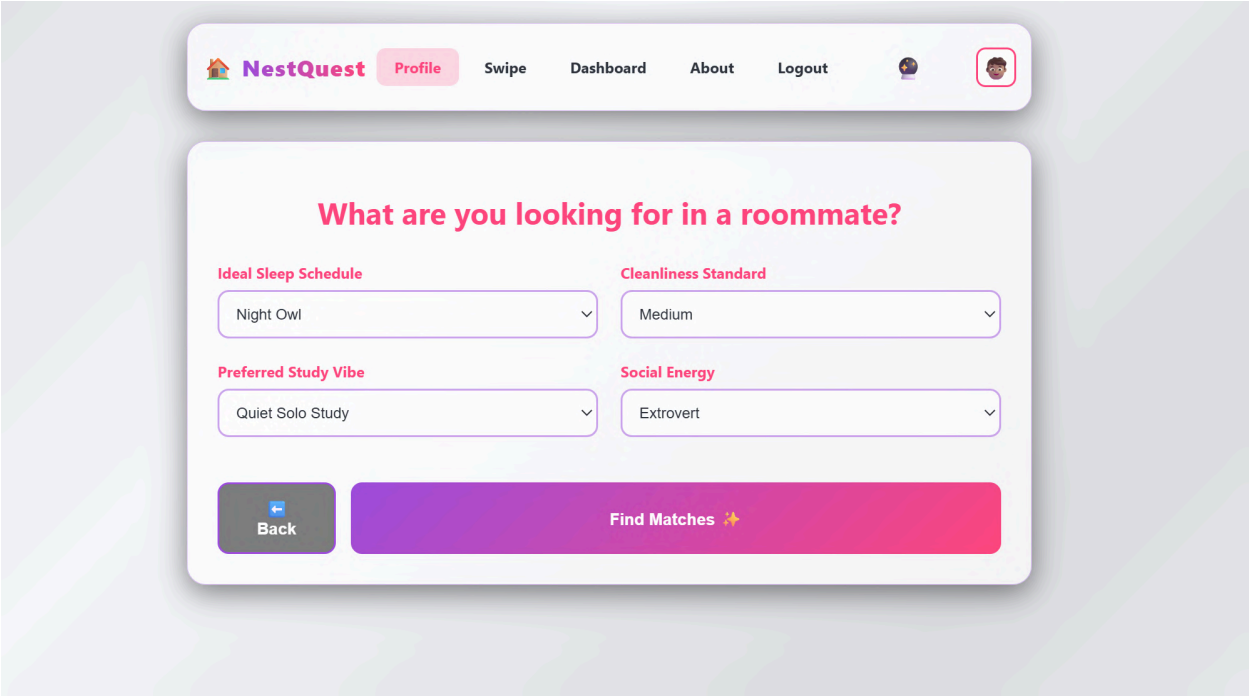
- **Design Implementation:** The UI provides real-time visual feedback. When a user drags a card, the card rotates and scales dynamically based on its distance from the center.
- **Technical Feature:** This interface is powered by an event-driven system tracking mousedown and mousemove coordinates. The "Match Percentage" visible on the card is generated by our weighted compatibility algorithm, which compares the user's stored preferences against the candidate's profile data.

7.3 Personalized Match Dashboard

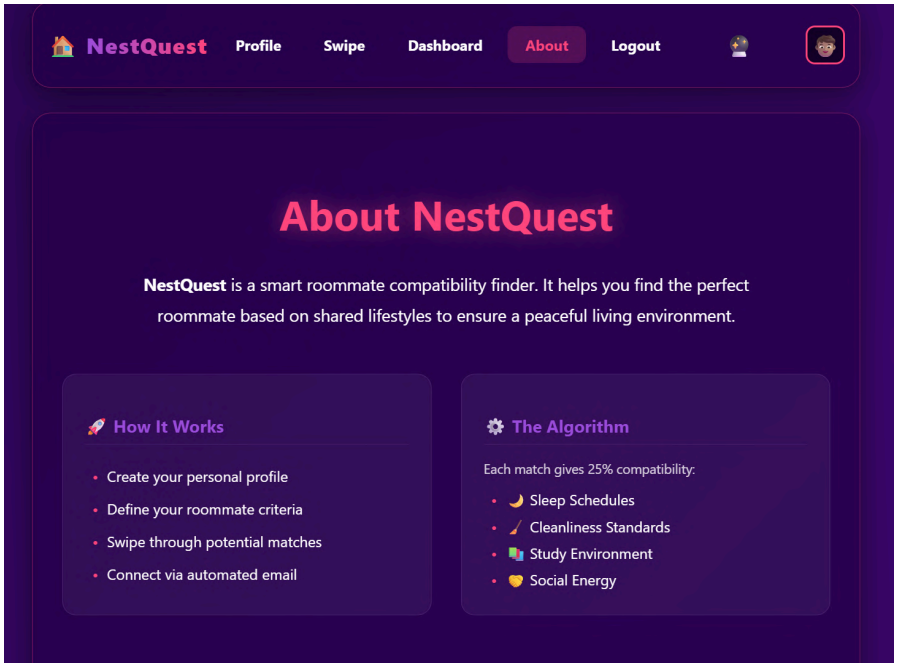


- **Design Implementation:** A clean, grid-based layout (CSS Grid) that organizes accepted matches. Each "Match Card" includes a quick-action "Contact" button.
- **Technical Feature:** The data displayed here is persistent. We utilize the LocalStorage API to retrieve the accepted_users array. The sorting dropdown seen at the top uses JavaScript's .sort() method to rearrange matches by compatibility or age in real-time without a page refresh.

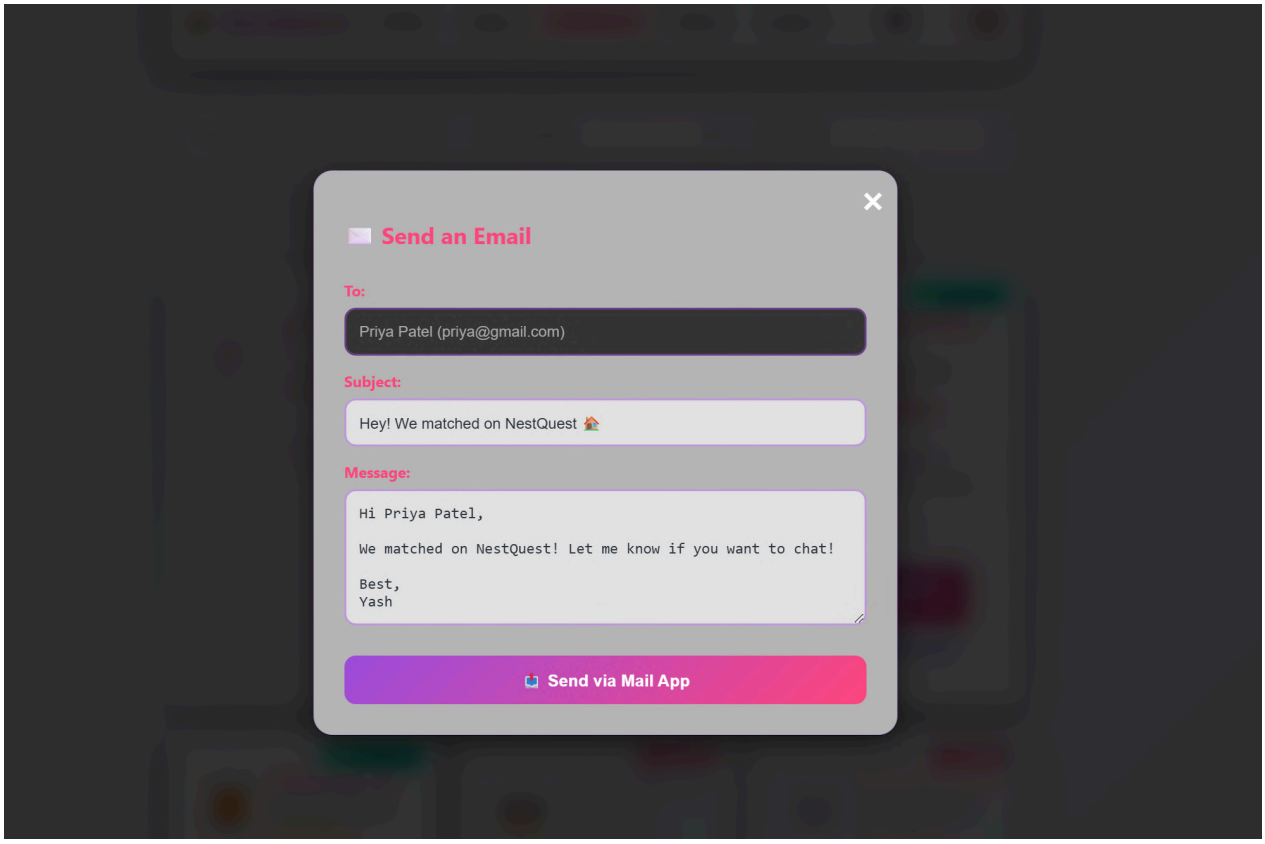
7.4 Theme Engine: Adaptive Light & Dark Modes



- **Design Implementation:** To ensure a premium user experience, we implemented a global theme engine.
- **Technical Feature:** Instead of hardcoding colors, we utilized CSS Custom Properties (Variables). The toggle button triggers a JavaScript function that adds a .light-theme class to the <body>, instantly swapping the variable values across the entire DOM for a seamless transition.



7.5 Communication Modal (Form Validation)



- **Technical Feature:** This represents the implementation of our third mandatory validated form. The contact modal incorporates strict client-side validation logic that actively prevents the submission of empty subjects or message bodies. Furthermore, to ensure seamless and secure external communication, the system utilizes the JavaScript `encodeURIComponent()` function. This safely sanitizes the user's input, parsing spaces and special characters, before dynamically passing the structured data payload to the user's native mail client via a custom `mailto:` protocol string

8. Conclusion & Future Scope

Through collaborative effort, detailed planning, and strict adherence to frontend web development principles, our team successfully built NestQuest. The project fulfills all technical requirements, including complex form validation, local data persistence, and a highly responsive, aesthetically pleasing interface.

Future Scope:

If this project were to be expanded in a future semester, we would migrate the dummy JSON database and LocalStorage architecture to a dedicated backend (such as Node.js/Express with MongoDB or Firebase) to allow for real-time, live user matchmaking and integrated WebSocket chatting.

Declaration: It is of utmost importance to state that all materials presented, coded, and submitted for this digital assignment are the original work of the team members listed on Page 1. The integrity of this deliverable has been fully maintained.